

[RI/2025/Q2]

2 Name your Jupyter Notebook as:

TASK2_server_<your name>_<index number>.ipynb

For each of the sub-tasks, add a comment statement at the beginning of the code using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 2.1  
        Program code
```

Output:

A company is developing a knowledge-based game that allows users to test their general knowledge through a five-question quiz. The system runs on a client-server model using socket programming. The server retrieves questions from a SQLite database, receives and validates user responses, calculates scores, and stores them. After the quiz, the server returns a sorted leaderboard to the client, displaying the top three performers. You are to implement core functionalities of this game system.

Task 2.1

Write a program that uses SQL to create a SQLite database named `quiz.db`. The database will be used to store questions from `questions.csv` and scores of quiz users.

Create a table `question` with the following fields:

- `id` (Integer, Primary Key)
- `question_text` (Text)
- `option_a` (Text)
- `option_b` (Text),
- `option_c` (Text),
- `option_d` (Text)
- `correct_option` (Text, storing 'A', 'B', 'C', or 'D')

Create another table `scores` with the following fields:

- `username` (Text, Primary Key)
- `score` (Integer)

The `scores` table will be populated later in Task 2.4.

Write a program to read quiz questions from a provided text file, `questions.csv` and insert this data into the `questions` table in your `quiz.db` database. [4]

[RI/2025/Q2 Cont']

Task 2.2

Declare a Python class named `Question` that encapsulates the data for a single quiz question.

It should have these attributes:

- `question_text`,
- `option_a`,
- `option_b`,
- `option_c`,
- `option_d`, and
- `correct_option`.

Include a constructor to initialise these attributes.

Create a method, `to_str()`, to return a single quiz question **as a string** and when printed, will be in this format:

```
What is 2 + 2?  
A. 3  
B. 4  
C. 5  
D. 6  
Your answer (A/B/C/D):
```

Create a method, `check_answer(userchoice)`, that when given a `userchoice`, returns `True`, if user guessed the correct option for the question. If `userchoice` is incorrect or not one of the four valid options (A/B/C/D), will return `False`. [3]

Task 2.3

Write a function `generate_5_questions()` that will:

- connect to the database, `quiz.db`,
- generate 5 unique questions
- returns a list of 5 `Question` objects.

Test your function. Use the `to_str()` method to print the contents of the 5 `Question` objects. [4]

[RI/2025/Q2 Cont ']

Task 2.4

Declare a Python class `QuizSession` that is used by the server to keep track of each new quiz attempt made by a user.

The `QuizSession` class contains the following attributes:

- `username`
- `score` (default to 0 as each quiz is a fresh attempt)

Implement the `updateScoresDB()` method:

- If this `username` does not exist in the `scores` table, insert a new `username` and his/her `score`, and returns a message: `"Your score has been updated on quiz.db"`
- If this `username` exists, it would mean that the player has already played the quiz before.
 - If the score attained in this `QuizSession` is higher than a previously attained score, update score and return a message: `"Congratulations, you have achieved a new high score"`,
 - Else, return the message: `"You did not outperform your previous score."`

You can assume that usernames are unique in the `scores` table.

[5]

Task 2.5

Write a function, `getLeaderBoard()` which queries the `scores` table and retrieves the Top 3 players by `score`, and returns a formatted string, breaking ties by sorting `username` alphabetically. Your function should **return a formatted string** in this format when printed:

```
Top 3 players
=====
Kelly: 5pts
Malcolm: 5pts
Joy: 4pts
```

[4]

[RI/2025/Q2 Cont ']

Task 2.6

Referring to the code in `client.ipynb`, build an iterative socket server code that:

- Accept a new client connection
- Request for username from the client
- Create a new `QuizSession` object for the `username`
- Sends 5 questions (1 at a time) to the client and receives answers.
- Validates answers and updates score in the `QuizSession` object
- Report final score to client
- Updates the `scores` table in `quiz.db` using `updateScoreDB()` method
- Sends client the leaderboard information, using `getLeaderboard()` function.
- Close connection with client

You are advised to make use of your code completed in Task 2.2 to 2.5 to complete the above tasks, when possible.

Run both server program and client programs in separate Jupyter notebooks, and save the output of the client-server interaction below the code cells. [10]